

Trees & Backtracking

Tree Hopping: <https://qoj.ac/problem/2905>

- You are given a tree and a permutation of all vertices. You need to check whether every consecutive pair in the permutation has tree distance at most 3.
 - Tree distance: shortest path in a tree between two nodes
- Hints:
 - Any root is OK!
 - We need to compute LCA: lowest common ancestor

```

N = int(input())
E = [[] for _ in range(N+1)]
for _ in range(N-1):
    A, B = map(int, input().split())
    E[A].append(B)
    E[B].append(A)
P = [int(input()) for _ in range(N)]

# Calculate parent and depth for each node
parent, depths = [-1] * (N+1), [0] * (N+1)
def visit(node, parent, depth):
    parents[node], depths[node] = parent, depth
    for child in E[node]:
        if child != parent: visit(child, node, depth + 1)
visit(1, -1, 0) # Root at node 1

# Calculate LCA
for i in range(len(P) - 1):
    x, y = P[i], P[i+1]
    dist = 0
    # Get both nodes to the same level
    while dist <= 3 and depths[x] > depths[y]:
        x = parents[x]; dist += 1
    while dist <= 3 and depths[y] > depths[x]:
        y = parents[y]; dist += 1
    # Find LCA by going the level up at the same time until the nodes meet
    while dist <= 3 and depths[x] != 0 and x != y:
        x = parents[x]; y = parents[y]; dist += 2
    if d > 3:
        print(0)
        break
else:
    print(1)

```

Balanced Tree Path: <https://qoj.ac/problem/9052>

- A directed path from one node to another is counted if the resulting character string is a valid bracket sequence.
- You need the total number of balanced paths. $n \leq 5000$.
- Key idea: n is small enough for one DFS starting from every node

```

PARENS = "()[]{}"

n = int(input())
char = [PARENS.index(c) for c in input()]
tree = [[] for _ in range(n)]
for _ in range(n - 1):
    u, v = (int(x) - 1 for x in input().split())
    tree[u].append(v)
    tree[v].append(u)

ans = 0
for root in range(n):
    if char[root] % 2 == 1: # check if it is closing parens
        continue

    # Non-recursive DFS
    parent = [-1 for _ in range(n)]
    stack = [root]
    seen = [False for _ in range(n)]
    seen[root] = True
    while stack:
        node = stack.pop()
        if char[node] % 2 == 0:
            cur = node
        elif parent[node] != -1 and char[node] - 1 == char[parent[node]]:
            cur = parent[parent[node]]
        else:
            continue
        ans += int(cur == -1) # reached the end of the stack!
        for v in tree[u]:
            if not seen[v]:
                seen[v] = True
                parent[v] = cur
                stack.append(v)

print(ans)

```



Problem A

10 Kinds of People

< Hide

The world is made up of 10 kinds of people, those who understand binary and those who do not. These different kinds of people do not always get along so well. Bob might ask for a 10000 ounce coffee (meaning binary) and Alice might make misinterpret his request as being in decimal and give him a 10011100010000 ounce coffee (binary). After Sue explains that this much coffee costs 100



Image by
[Christiaan Colen](#)

Floodfill: 10 kinds of people

- <https://purdue.kattis.com/courses/CS211-CP1/2024-Spring/assignments/xazco7/problems/10kindsofpeople>
- Also see <https://open.kattis.com/contests/naipc17-p08/problems/gold>
- Use DFS of BFS (also known as floodfill)

```

from deque import deque

R, C = map(int, input().split())
grid = [input() for _ in range(R)]
colour = [[-1 for _ in range(C)] for _ in range(R)]
area = 0
for r in range(R):
    for c in range(C):
        if colour[r][c] == -1:
            q = deque([(r, c)]) # queue in other languages
            colour[r][c] = area
            while len(q):
                cr, cc = q.popleft()
                for dr, dc in [(-1, 0), (0, -1), (0, 1), (1, 0)]:
                    nr, nc = cr + dr, cc + dc
                    if (
                        # valid coordinates
                        nr >= 0 and nc >= 0 and nr < R and nc < C
                        # not visited
                        and colour[nr][nc] == -1
                        # same symbol
                        and grid[nr][nc] == grid[r][c]
                    ):
                        colour[nr][nc] = area
                        q.append((nr, nc))
            area += 1

N = int(input())
for r in range(N):
    r1, c1, r2, c2 = map(lambda x: int(x) - 1, input().split())
    if colour[r1][c1] == colour[r2][c2]:
        print("binary" if grid[r1][c1] == '0' else "decimal")
    else:
        print("neither")

```

Tries

- https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&category=24&page=show_problem&problem=585

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False

def insert(root, s):
    node = root
    for ch in s:
        if node.end: # some node ended here: prefix!
            return False
        if ch not in node.children:
            node.children[ch] = TrieNode()
        node = node.children[ch]
    if node.end or node.children: # still a prefix...
        return False
    node.end = True
    return True

root, decodable = TrieNode(), True
group = 1
for line in input():
    s = line.strip()
    if s == "9":
        print(f"Set {group} is{' ' if decodable else ' not'} immediately decodable")
        group += 1
        root, decodable = TrieNode(), True
    else:
        if decodable and not insert(root, s):
            decodable = False
```